信息安全综合实验课程实验报告

实验五：基于 Detours 技术的 Windows 系统行为监控实验

学　　院　密码与网络空间安全学院

专　　业　信息安全、法学双学位班

学　　号　2313815

姓　　名　段俊宇

班　　级　信息安全、法学双学位班

一、实验目的

1. 熟悉微软发布的 Detours（绕行道）技术，理解 Detours 挂钩系统 API 调用的原理。
2. 了解基于 Detours 技术的无文件 Ring3 Rootkit 工具 R77，能够安装并使用 R77 隐藏 Windows 系统上的进程、文件、注册表项、网络连接。
3. 基于 Detours 和 R77 开发一套 Windows 系统行为监控工具，能够开发记录当前系统的操作行为，例如文件操作、进程创建、注册表操作、网络连接等。

二、实验原理

2.1 Detours（绕行道）技术

Detours 是微软发布的一个函数库，提供了简单、高效、稳定的 API 挂钩功能。Detours 可以用于在 ARM, ARM64, X86, X64 和 IA64 CPU 平台上，拦截应用程序中的 Win32 API 调用，比如添加调试功能、拦截特定的系统信息。Detour 的官方代码及文档详见 https://www.microsoft.com/en-us/research/project/detours/Detours。

Detours 提供的函数可以重写内存中的代码，将目标函数的前几个指令替换为无条件跳转指令，将 CPU 控制权转移到指定的 Detour 函数。Detours 还可以编辑二进制 DLL 文件的导入表，将任意数据段附加到现有二进制文件以及将 DLL 加载到新进程内存空间中。Detours 技术的实现过程如下：

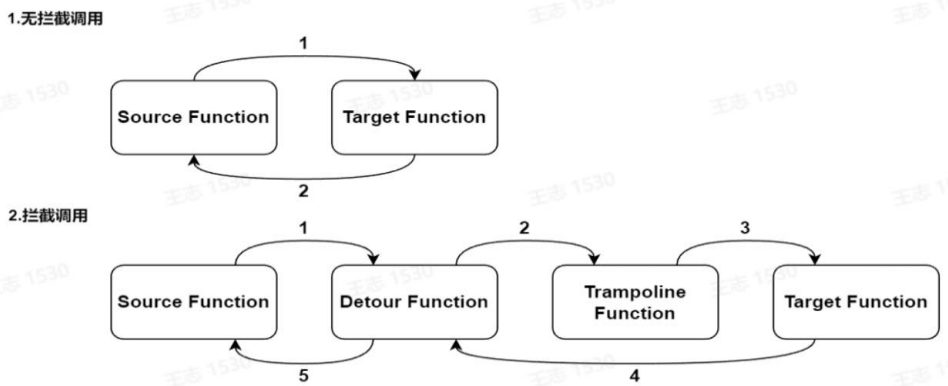


图 1 Detours 拦截函数调用流程

2.1.1 C++调用 Detours

要使用 C++调用 Detours 库，首先要对 Detours 库的源码进行编译，得到库文件。从代码仓库中下载源码，解压后 src 文件夹中即为需要编译的源码文件。

图 2 Detours 源码

1. **使用 nmake 命令编译**

以 VS2019 环境为例，打开 VS 开发人员命令行窗口，进入 Detours 源码文件目录中的 src 目录，输入命令“nmake”进行编译。

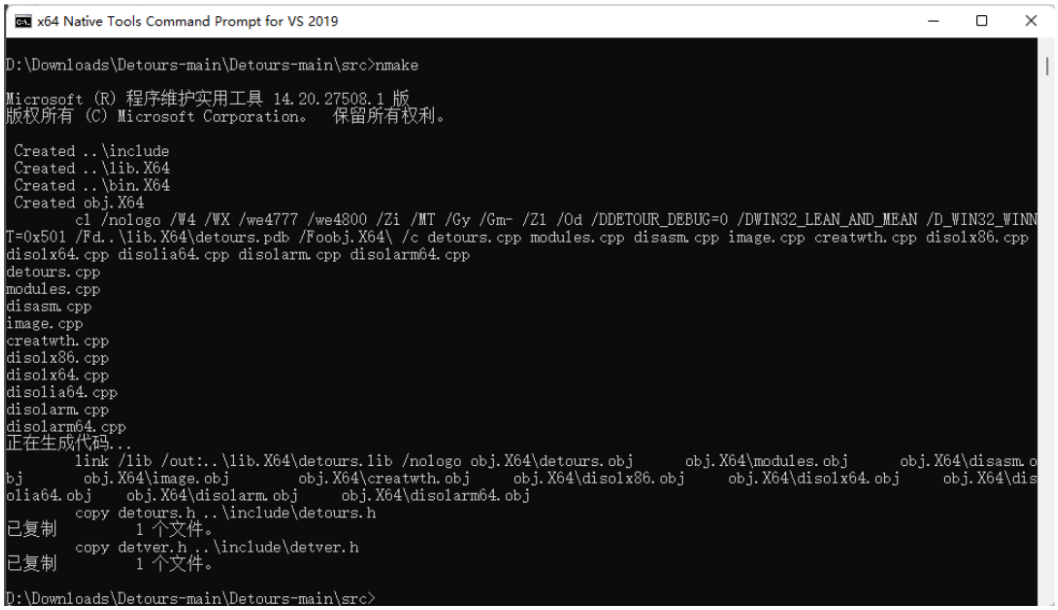


图 3 nmake 指令编译

2. **导入库文件**

将形成的 detours.h、detver.h 和 detours.lib 文件放入对应程

序目录下，导入即可调用。

2.1.2 Detours 中的重要函数

1. DetourAttach

- 函数功能：截获目标函数。
- 函数定义：

```
LONG DetourAttach(
    _Inout_ PVOID * ppPointer,
    _In_    PVOID pDetour
);
```

- 参数：

  ppPointer ： 指向将要被截获函数地址的函数指针。

  pDetour ： 指向实际运行的函数的指针。

- 返回值：执行成功，则返回 NO_ERROR；否则，返回错误代码。

2. Detour Detach

- 函数功能：解除截获目标函数。
- 函数定义：

```
LONG DetourDetach(
    _Inout_ PVOID * ppPointer,
    _In_    PVOID pDetour
);
```

- 参数：

  ppPointer ： 指向将要被接触截获函数地址的函数指针。

  pDetour ： 指向实际运行的函数的指针。

- 返回值：执行成功，则返回 NO_ERROR；否则，返回错误代码。

3. DetourTransactionBegin

- 函数功能：开始一次截获/解除截获。
- 函数定义：

```
LONG DetourTransactionBegin(VOID);
```

- 参数：无
- 返回值：执行成功，则返回 NO_ERROR；否则，返回错误代码。

4. DetourUpdateThread

- 函数功能：列入一个在 Detour 事务过程中要更新的线程。
- 函数定义：

```
LONG DetourUpdateThread(
    _In_ HANDLE hThread
);
```

- 参数：

  hThread：要用挂起的事务更新的线程的句柄。如果 hThread 等于当前线程伪句柄（由 GetCurrentThread（）返回），则不执行任何操作。
- 返回值：执行成功，则返回 NO_ERROR；否则，返回错误代码。

5. DetourTransactionCommit

- 函数功能：执行当前的捕获/解除捕获事务。
- 函数定义：

```
LONG DetourTransactionCommit(VOID);
```

- 参数：无
- 返回值：执行成功，则返回 NO_ERROR；否则，返回错误代码。

2.2 R77 Rootkit 介绍

R77-Rootkit 是基于 Detours 技术开发的一款开源的功能强大的无文件 Ring3 Rootkit，并且带有完整的安全工具和持久化机制，可以实现进程、文件和网络连接等操作及任务的隐藏。 R77 的源代码在 GitHub 上，下载地址为 https://github.com/bytecode77/r77-rootkit。R77 Rootkit 将所有以“$77”为前缀命名的实体都将被隐藏，动态配置系统允许广大研究人员通过 PID 来隐藏进程，通过完整路径来隐藏文件系统，或通过指定端口隐藏 TCP&UDP 连接，如图 2 所示。

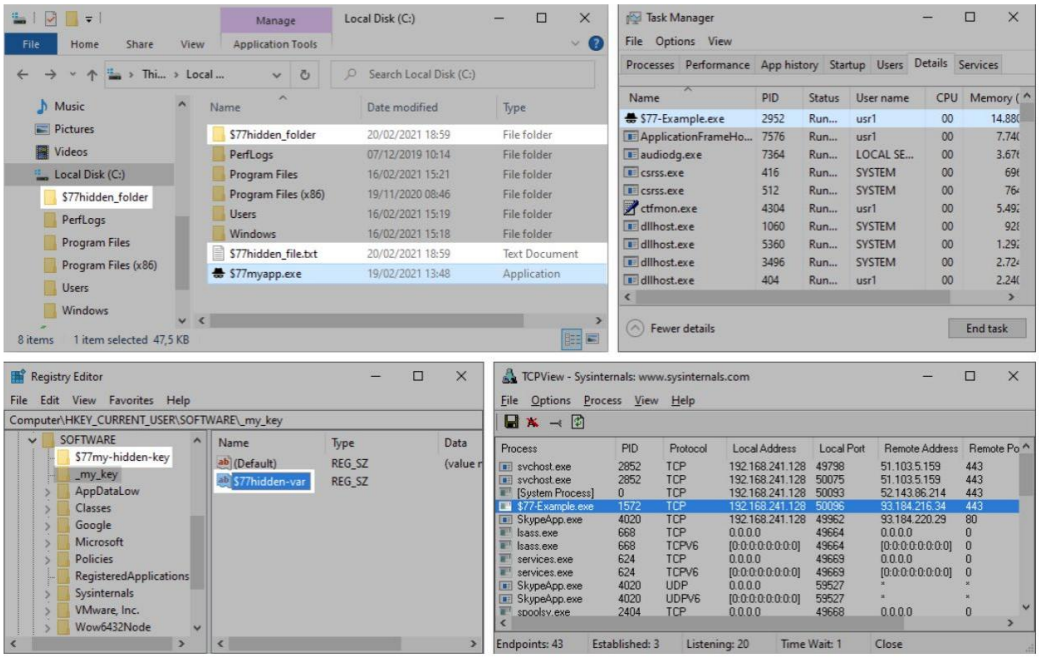


图 2 R77 Rootkit 隐藏系统文件、进程、注册表值、网络链接

2.2.1 R77 Rootkit 系统配置

R77 Rootkit 动态配置系统允许广大研究人员通过 PID 来隐藏进程，通过完整路径来隐藏文件系统，或通过指定端口隐藏 TCP&UDP 连接，如图 3 所示。配置信息存储在“HKEY_LOCAL_MACHINE\SOFTWARE\$77config”中，并且可以在未提权状态下由任何进程写入。这个键的 DACL 被设置为可以给任意用户授予完整访问权。“$77config”键在注册表编辑器被注入了 Rootkit 之后会自动隐藏。

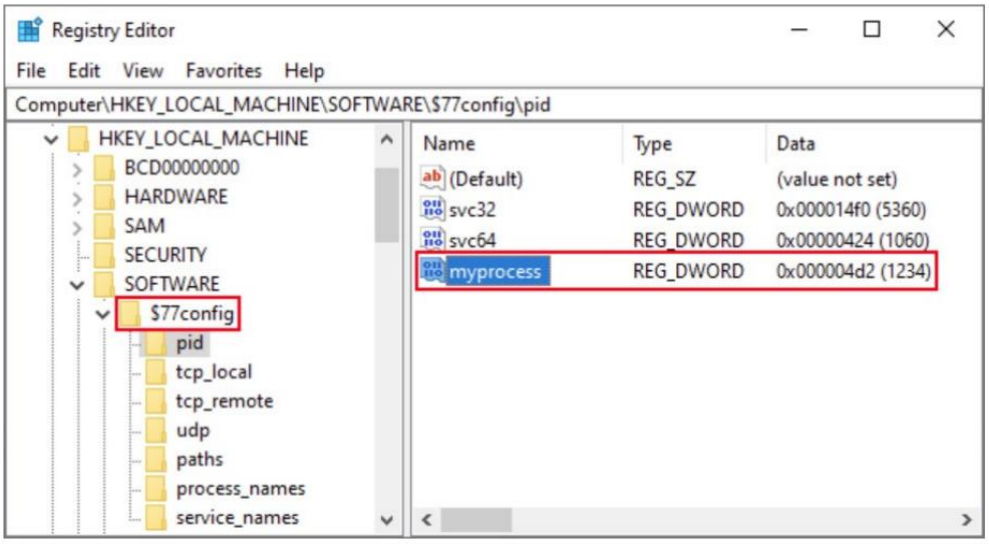


图 3 R77 Rootkit 配置

2.2.2 R77 Rootkit 的安装

R77 Rootkit 可以直接使用 GitHub 代码库中“Install.exe”进行安装，安装工具会将 R77 服务在用户登录之前开启，后台进程会向所有当前正在运行以及后续生成的进程中注入命令。这里需要使用两个进程来分别注入 32 位和 64 位进程，这两个进程都可以使用配置系统和 PID 来进行隐藏。“Uninstall.exe”程序负责将 r77 从系统中卸载掉，并解除 Rootkit 跟所有进程的绑定关系。

2.2.3 R77 无文件持久化

Rootkit 将驻留在系统内存中，不会将任何文件写入磁盘，这种机制是分多个阶段实现的。

（1）阶段一

安装程序为 32 位和 64 位 r77 服务创建两个计划任务。计划任务确实需要存储名为$77svc32.job 和$77svc64.job 的文件，这是无文件概念的唯一例外。但是，一旦 Rootkit 运行，计划任务也会通过前缀隐藏，如图 4 所示。

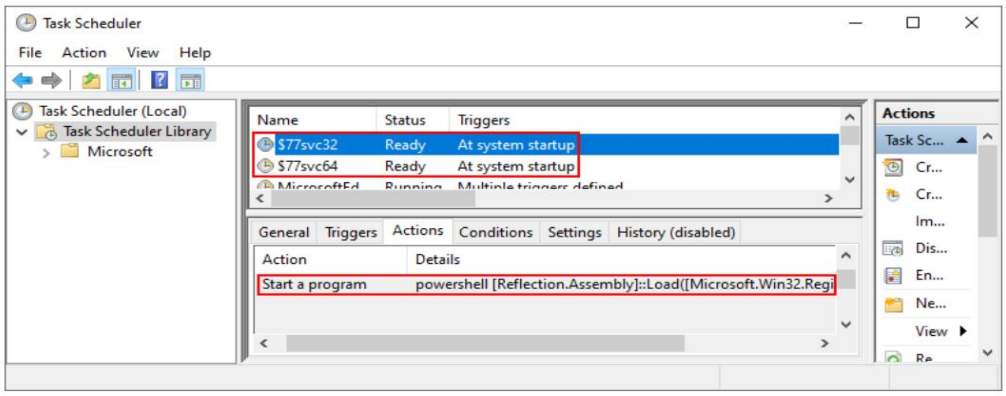


图 4 进程隐藏

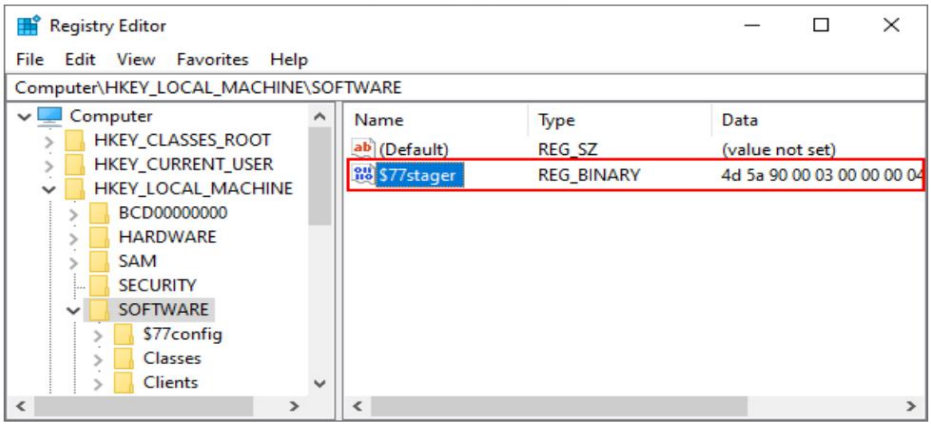


图 5 注册表修改

计划任务将使用下列命令开启“powershell.exe”：

[Reflection.Assembly]::Load([Microsoft.Win32.Registry]::LocalMachine.OpenSubkey('SOFTWARE').GetValue('$77stager')).EntryPoint.Invoke($Null,$Null)

该命令是内联的，不需要.ps1 脚本。这里，使用 PowerShell 的.NET Framework 功能从注册表加载 C#可执行文件并在内存中执行。由于命令行的最大长度为 260（MAX_PATH），因此只有足够的空间执行简单的 Assembly.Load().EntryPoint.Invoke()，如图 5 所示。

（2）阶段二

执行的 C#代码为 stager，它将会使用 Process Hollowing 技术创建 R77 服务进程。R77 服务是一个本地可执行文件，分别以 32 位和 64 位架构分别编译。父进程被设置为了 winlogon.exe 以增加欺骗性（模糊性）。另外，这两个进程被 ID 隐藏，在任务管理器中不可见。磁盘上从未存储可执行文件或 DLL。stager 存储在注册表中，并从其资源加载 r77 服务可执行文件。

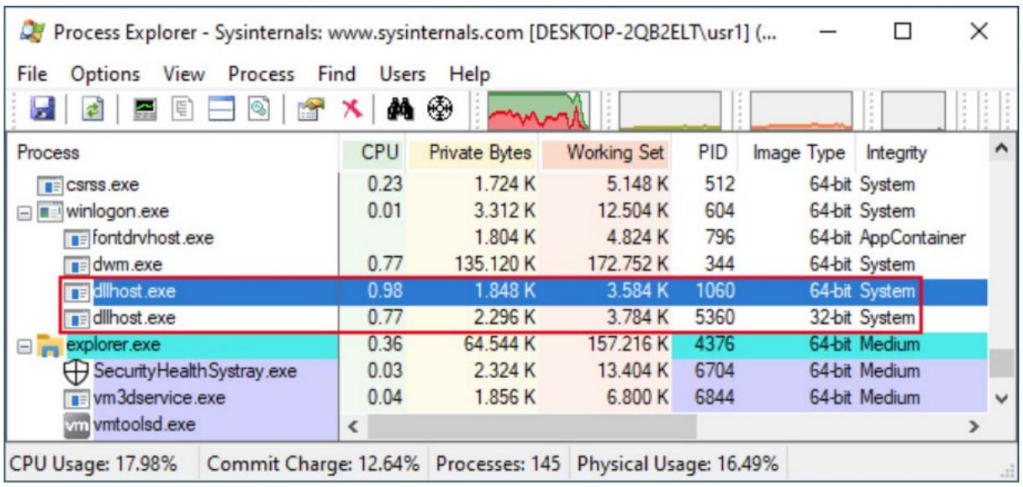


图 6 进程隐藏

R77 测试控制台可以用来向单独进程注入 r77，或接触进程跟 Rootkit 的绑定关系。

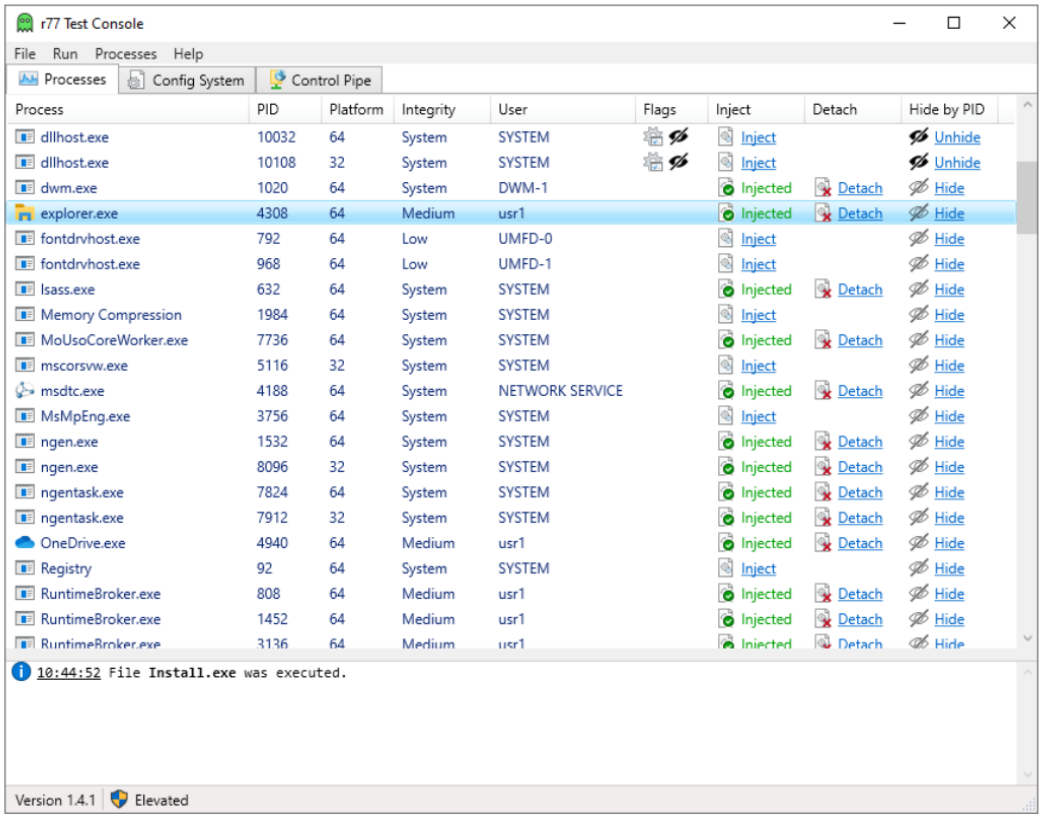


图 7 R77 测试控制台

三、实验环境

Windows 操作系统，Detours 函数库，R77 Rootkit 工具源代码。R77 工具下载地址：https://downloads.bytecode77.com/r77Rootkit%201.4.2.zip（解压密码：bytecode77）。R77 的技术文档下载地址：https://docs.bytecode77.com/r77-rootkit/Technical%20Documentation.pdf

四、实验过程

1. 使用 Detours 库，实现对 WindowsAPI 函数进行挂钩 Hook

首先按照上面步骤对 Detours 库进行配置，然后编写如下所示的代码，对 CreateFileA 函数进行挂钩。

```
#define _CRT_SECURE_NO_WARNINGS
#include <windows.h>
#include <detours.h>
#include <stdio.h>

#pragma comment(lib,"detours.lib")
```

```
7.
8. // 原函数指针
9. HANDLE(WINAPI* TrueCreateFileA)(
10.     LPCSTR lpFileName,
11.     DWORD dwDesiredAccess,
12.     DWORD dwShareMode,
13.     LPSECURITY_ATTRIBUTES lpSecurityAttributes,
14.     DWORD dwCreationDisposition,
15.     DWORD dwFlagsAndAttributes,
16.     HANDLE hTemplateFile
17.     ) = CreateFileA;
18.
19.
20. // Hook 函数
21. HANDLE WINAPI MyCreateFileA(
22.     LPCSTR lpFileName,
23.     DWORD dwDesiredAccess,
24.     DWORD dwShareMode,
25.     LPSECURITY_ATTRIBUTES lpSecurityAttributes,
26.     DWORD dwCreationDisposition,
27.     DWORD dwFlagsAndAttributes,
28.     HANDLE hTemplateFile
29. )
30. {
31.     FILE* fp = fopen("log.txt", "a");
32.
33.     if (fp)
34.     {
35.         fprintf(fp, "CreateFile called: %s\n", lpFileName);
36.         fclose(fp);
37.     }
38.
39.     return TrueCreateFileA(
40.         lpFileName,
41.         dwDesiredAccess,
42.         dwShareMode,
43.         lpSecurityAttributes,
44.         dwCreationDisposition,
45.         dwFlagsAndAttributes,
46.         hTemplateFile
47.     );
48. }
49.
50.
```

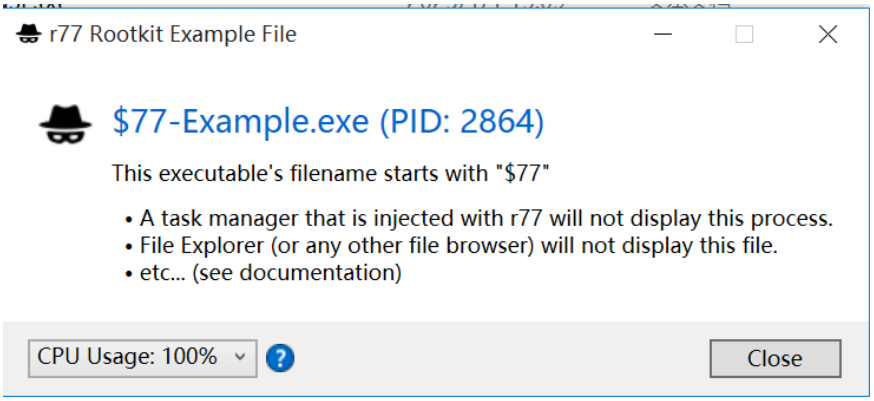
```
51. void Hook()
52. {
53.     DetourTransactionBegin();
54.     DetourUpdateThread(GetCurrentThread());
55.
56.     DetourAttach(&(PVOID&)TrueCreateFileA, MyCreateFileA);
57.
58.     DetourTransactionCommit();
59. }
60.
61. void UnHook()
62. {
63.     DetourTransactionBegin();
64.     DetourUpdateThread(GetCurrentThread());
65.
66.     DetourDetach(&(PVOID&)TrueCreateFileA, MyCreateFileA);
67.
68.     DetourTransactionCommit();
69. }
70.
71. int main()
72. {
73.     Hook();
74.
75.     // 测试
76.     HANDLE hFile = CreateFileA(
77.         "test.txt",
78.         GENERIC_WRITE,
79.         0,
80.         NULL,
81.         CREATE_ALWAYS,
82.         FILE_ATTRIBUTE_NORMAL,
83.         NULL
84.     );
85.
86.     if (hFile != INVALID_HANDLE_VALUE)
87.     {
88.         CloseHandle(hFile);
89.     }
90.
91.     UnHook();
92.
93.     return 0;
94. }
```

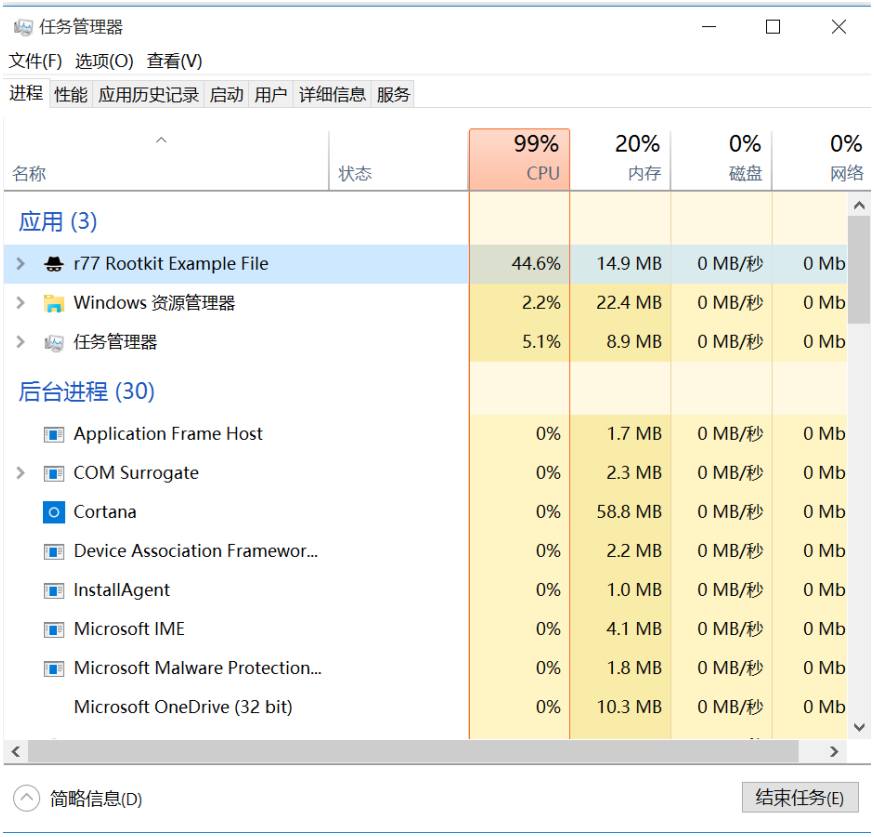
运行上述代码后发现项目文件夹下多出了 log.txt 和 test.txt，log.txt 内容为 CreateFile called:test.txt。这说明成功对 CreateFile 挂钩，并且没有影响函数的正常执行。

2. R77 Rootkit 工具隐藏实验

（1） 指定进程隐藏

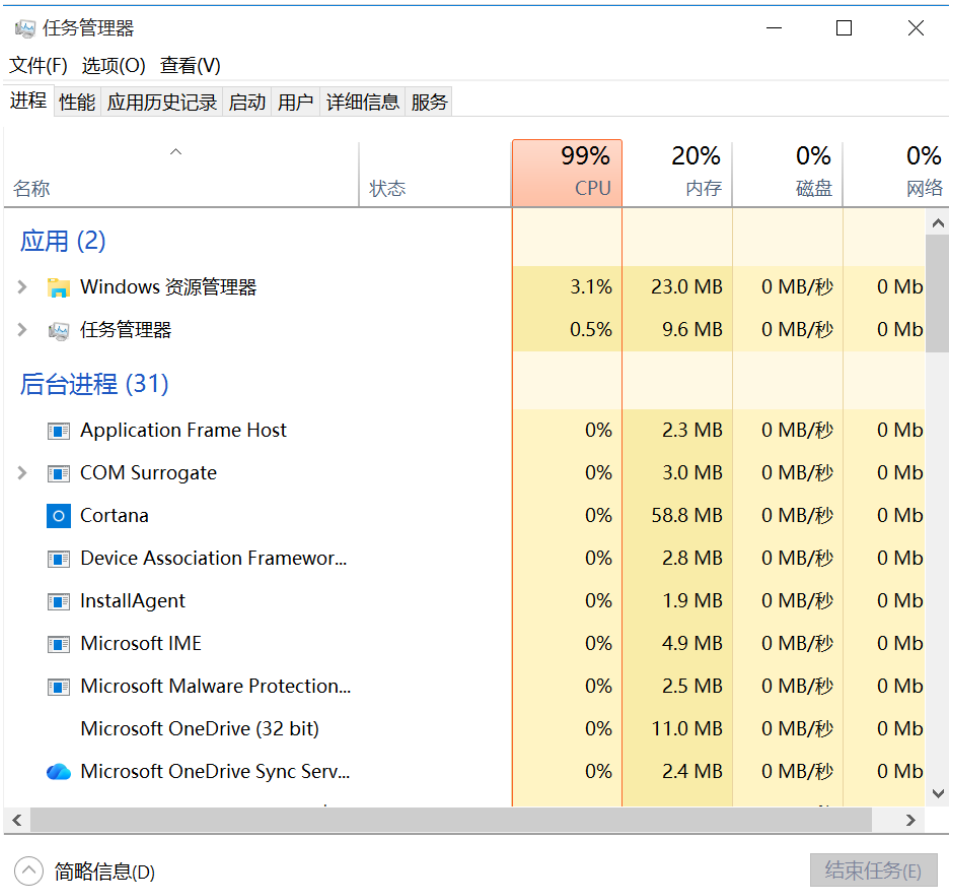
首先运行$77-Example.exe，将 CPU 占用率调整成 100%，这样可以更清晰的看出是否发生变化。打开任务管理器，发现 CPU 占用率是 99%，并且$77-Example.exe 也在上面显示



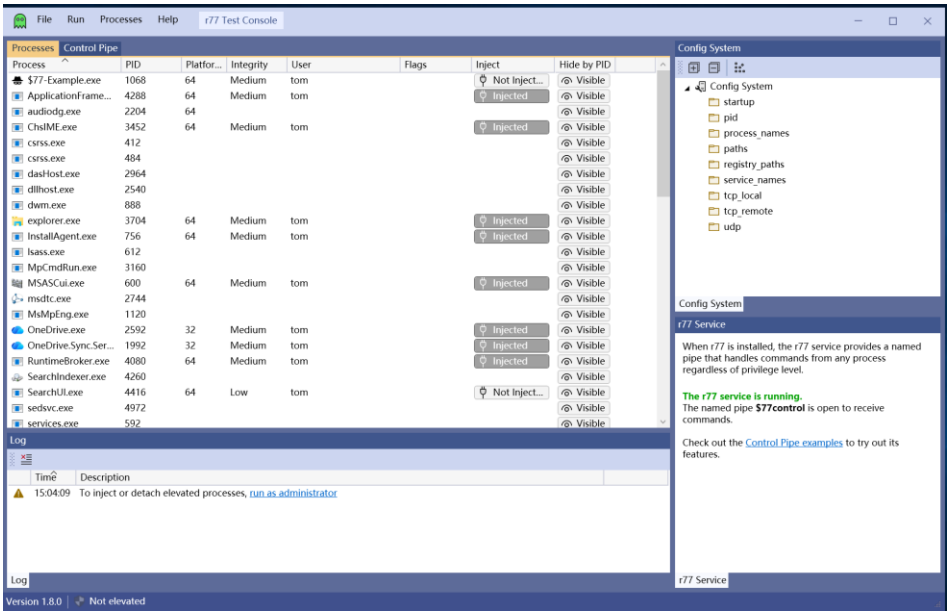


$77-Example.exe 的 CPU 占用率一直在变化，但是都持续在 75%以上。

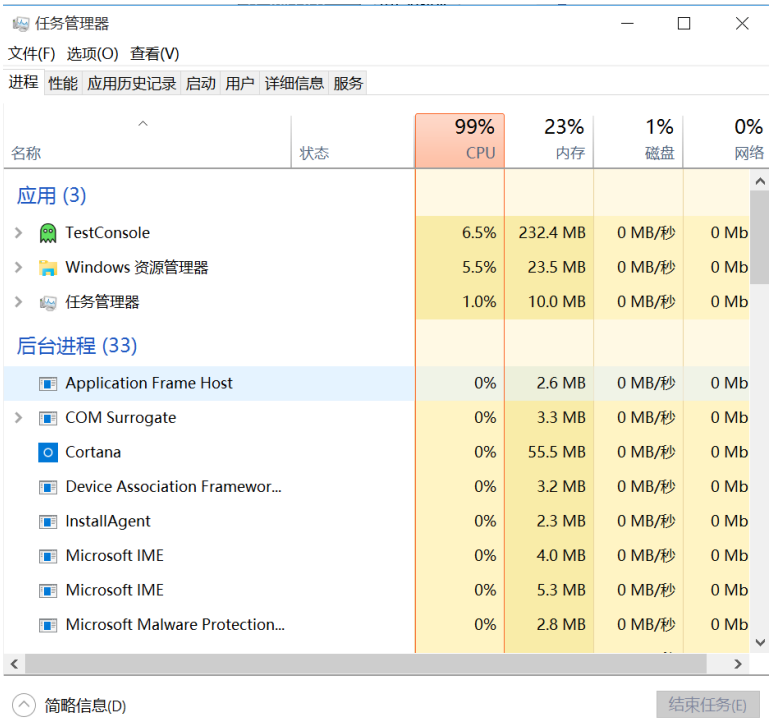
之后运行 install.exe，将这一进程隐藏后，任务管理器如下图所示：



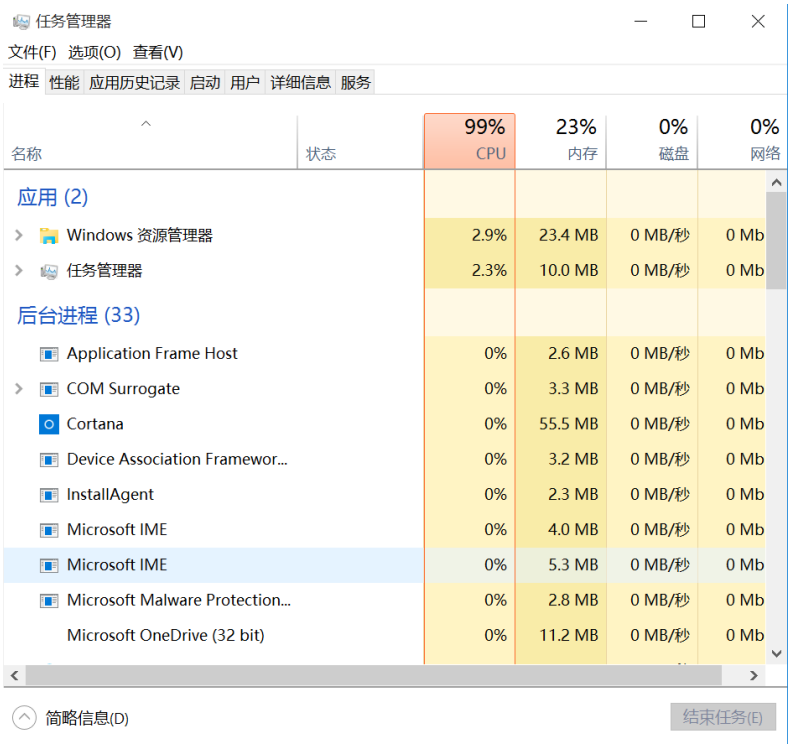
在虚拟机的任务栏中也能看到$77-Example.exe，CPU 占用率仍然是 99%，这说明该进程并未结束，而是被隐藏起来了。接下来打开 TestConsole.exe，这个是隐藏进程的控制台，在里面可以看到$77-Example.exe 依然在运行，验证了上面的结论。



此时在任务管理器中可以看到多了 TestConsole.exe 的进程，如下图所示：

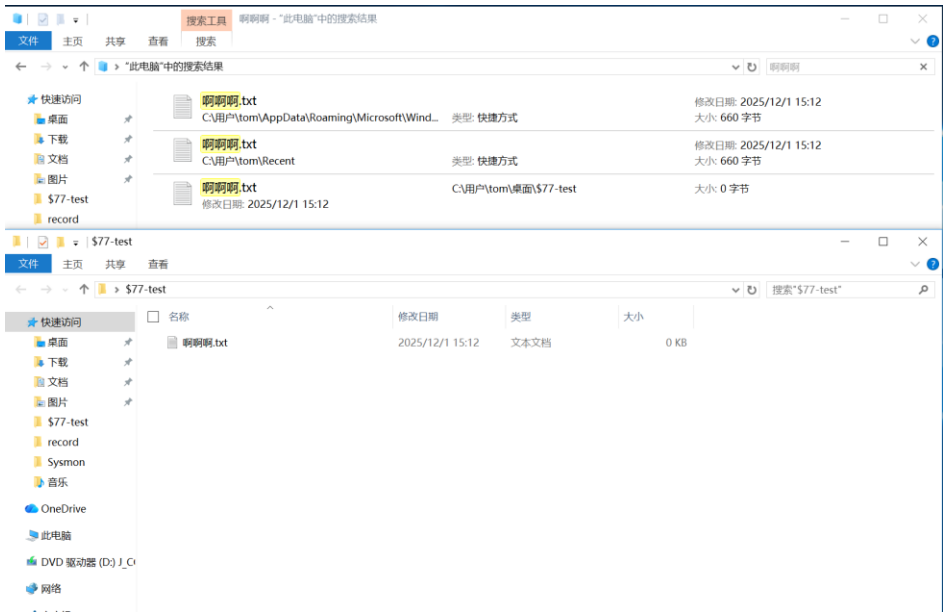


然后隐藏指定的进程，在 TestConsole.exe 控制台中将自己的进程从 Visible 改成 hidden，然后刷新任务管理器，会发现 TestConsole.exe 之前的进程也不见了，说明成功验证了进程隐藏功能。

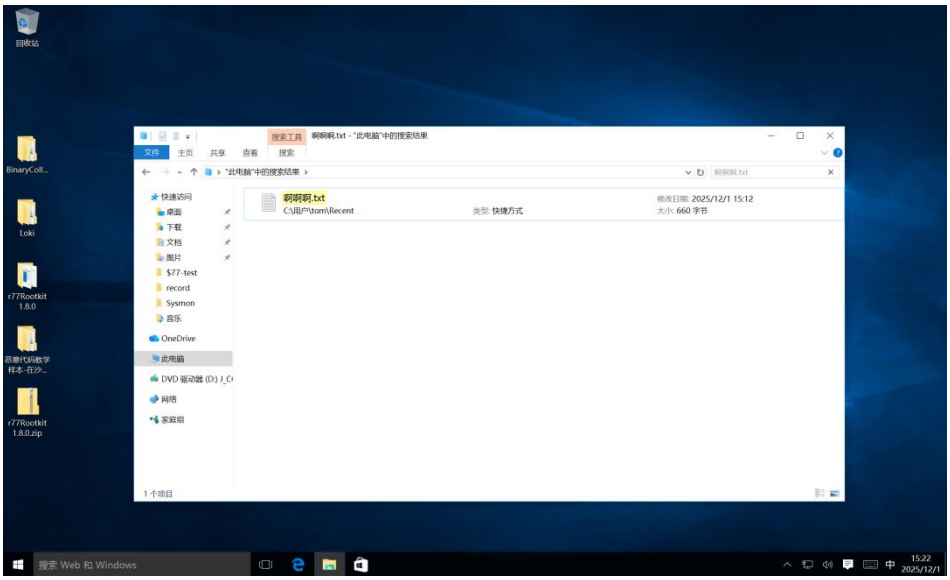


（2）文件的隐藏

下面验证文件隐藏功能，首先在桌面创建一个文件夹，名字需要 $77 开头，然后在里面创建一个 txt 文档作为测试。使用文件资源管理器能够搜索到该文件，如下所示：



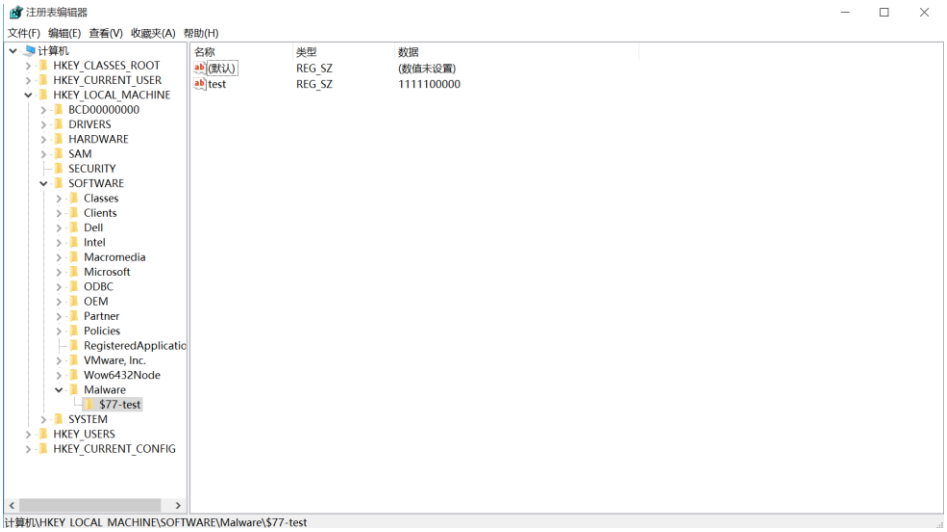
接下来运行 install.exe，发现桌面上的文件夹消失了，而且在文件资源管理器中搜索时，无法找到路径为 C:\用户\tom\桌面\$77-test\啊啊啊.txt 的文件，而 Recent 里面记录了最近的文件，所以会有这个 txt 文件，但是$77-test 文件夹无法搜索到，说明成功验证了文件隐藏功能。



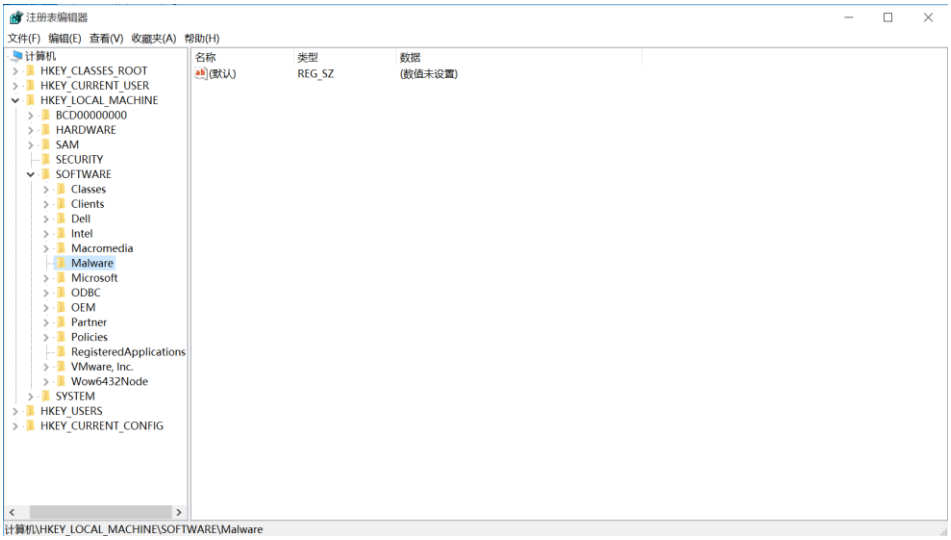
（3）注册表的隐藏

下面验证注册表隐藏功能，首先在 HKEY_LOCAL_MACHINE\SOFTWARE 中

新创建 Malware\$77-test，并设置键值 test 为 1111100000。

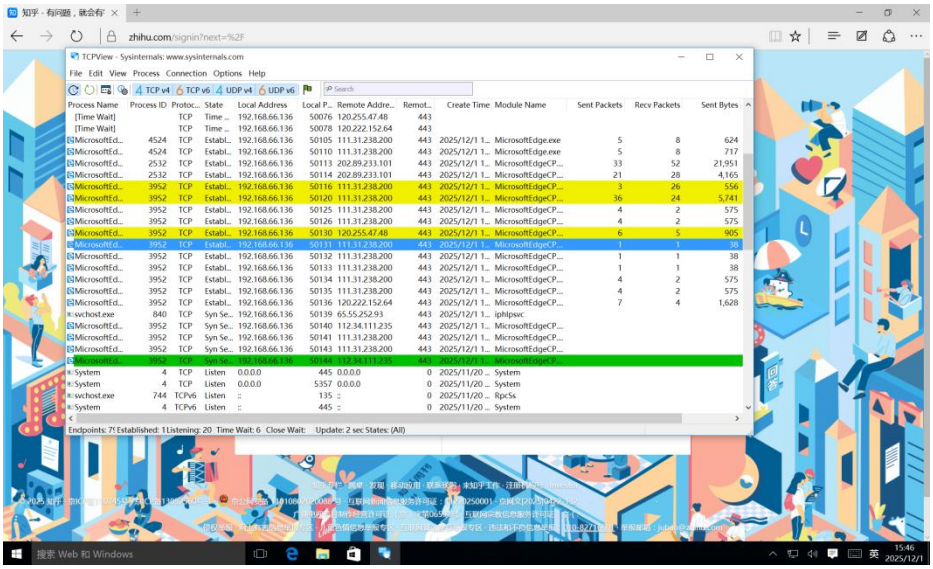


运行 install.exe，再次打开注册表发现这个注册表项已经消失了，说明成功验证了注册表隐藏功能。

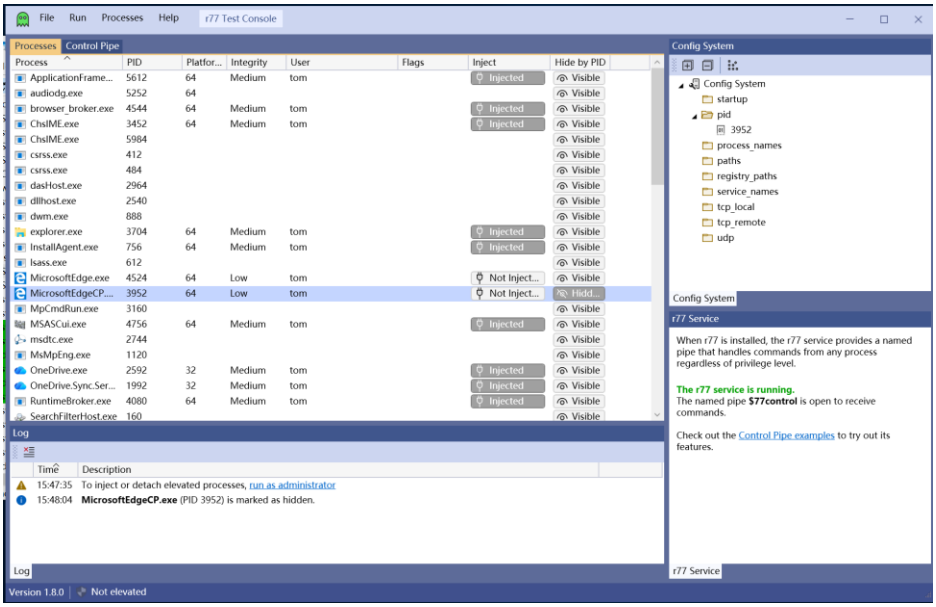


（4）网络连接的隐藏

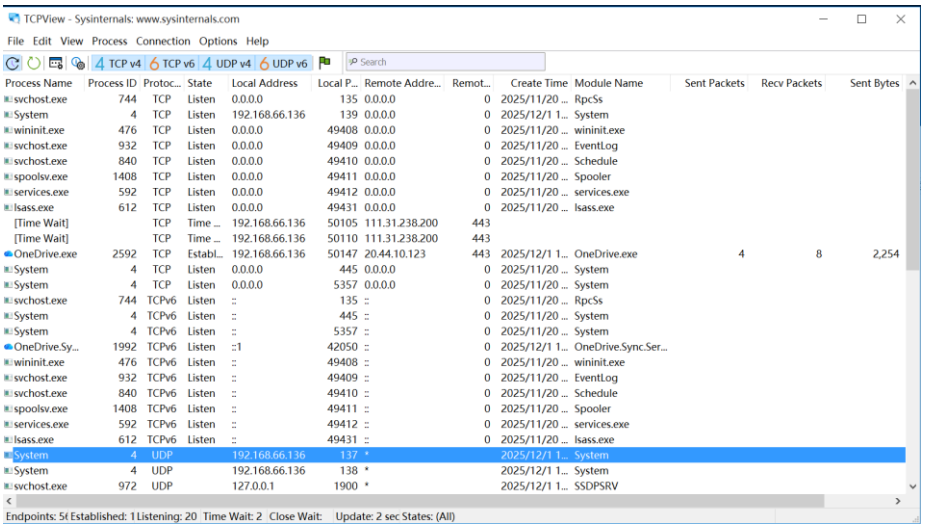
下面验证网络连接隐藏功能，首先需要下载 TCPView 这个工具来查看目前存在的 TCP 和 UDP 连接。之后在浏览器中随便搜索一个网址，这里我搜索 zhihu.com，然后打开 TCPView，刷新网页，能够发现该 TCP 连接对应的进程 UID 是 3952。



运行 install.exe，然后打开 TestConsole.exe，发现这一进程在里面有显示，接着将 Visible 调成 hidden，如下图所示：



再回到 TCPView 中，发现该进程消失了，但是网页依然开着，说明被隐藏了。之后我对 UDP 连接也进行了验证，同样隐藏了 UDP 进程，说明成功验证网络连接隐藏功能。



这样就成功验证了 R77 恶意软件的隐藏功能！

3. 基于 Detours 和 R77 Rootkit 的 Windows 行为监控工具

为了实现对 Windows 行为监控，需要编译一个 DLL 文件和一个注入器 injector。首先注入器注入进程，然后在注入器中调用 DLL 文件并监控相关操作和 API 函数的调用。如下是 DLL 文件的代码：

```
// FileMonitorDLL.cpp
#define _CRT_SECURE_NO_WARNINGS
#include <windows.h>
#include <detours.h>
#include <stdio.h>

#pragma comment(lib,"detours.lib")

// 原始函数指针 - CreateFileA
HANDLE(WINAPI* TrueCreateFileA)(
    LPCSTR lpFileName,
    DWORD dwDesiredAccess,
    DWORD dwShareMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD dwCreationDisposition,
    DWORD dwFlagsAndAttributes,
    HANDLE hTemplateFile
    ) = CreateFileA;

// 原始函数指针 - CopyFileA
BOOL(WINAPI* TrueCopyFileA)(
    LPCSTR lpExistingFileName,
    LPCSTR lpNewFileName,
```

```
    BOOL bFailIfExists
    ) = CopyFileA;

// 原始函数指针 - DeleteFileA
BOOL(WINAPI* TrueDeleteFileA)(
    LPCSTR lpFileName
    ) = DeleteFileA;

// 原始函数指针 - MoveFileA
BOOL(WINAPI* TrueMoveFileA)(
    LPCSTR lpExistingFileName,
    LPCSTR lpNewFileName
    ) = MoveFileA;

// 日志锁
CRITICAL_SECTION g_LogLock;
BOOL g_bLogInit = FALSE;

// 写日志函数
void WriteLog(const char* operation, const char* details)
{
    if (!g_bLogInit) return;

    EnterCriticalSection(&g_LogLock);

    FILE* fp = fopen("behavior_log.txt", "a");
    if (fp)
    {
        fprintf(fp, "[%s] %s\n", operation, details);
        fprintf(fp, "----------------------------\n");
        fclose(fp);
    }

    LeaveCriticalSection(&g_LogLock);
}

// Hook CreateFileA
HANDLE WINAPI MyCreateFileA(
    LPCSTR lpFileName,
    DWORD dwDesiredAccess,
    DWORD dwShareMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD dwCreationDisposition,
    DWORD dwFlagsAndAttributes,
```

```
    HANDLE hTemplateFile
)
{
    char logMsg[512];
    const char* action = "OPEN";

    if (dwCreationDisposition == CREATE_NEW || dwCreationDisposition == CREATE_ALWAYS)
        action = "CREATE";

    sprintf(logMsg, "文件: %s, 操作: %s, 访问类型: %s%s",
        lpFileName, action,
        (dwDesiredAccess & GENERIC_READ) ? "READ " : "",
        (dwDesiredAccess & GENERIC_WRITE) ? "WRITE" : "");

    WriteLog("FileAccess", logMsg);

    return TrueCreateFileA(lpFileName, dwDesiredAccess, dwShareMode,
        lpSecurityAttributes, dwCreationDisposition,
        dwFlagsAndAttributes, hTemplateFile);
}

// Hook CopyFileA
BOOL WINAPI MyCopyFileA(
    LPCSTR lpExistingFileName,
    LPCSTR lpNewFileName,
    BOOL bFailIfExists
)
{
    char logMsg[512];
    sprintf(logMsg, "从: %s 复制到: %s", lpExistingFileName, lpNewFileName);
    WriteLog("CopyFile", logMsg);

    return TrueCopyFileA(lpExistingFileName, lpNewFileName, bFailIfExists);
}

// Hook DeleteFileA
BOOL WINAPI MyDeleteFileA(LPCSTR lpFileName)
{
    char logMsg[512];
    sprintf(logMsg, "删除文件: %s", lpFileName);
    WriteLog("DeleteFile", logMsg);

    return TrueDeleteFileA(lpFileName);
```

```
}

// Hook MoveFileA
BOOL WINAPI MyMoveFileA(LPCSTR lpExistingFileName, LPCSTR lpNewFileName)
{
    char logMsg[512];
    sprintf(logMsg, "从: %s 移动到: %s", lpExistingFileName, lpNewFileName);
    WriteLog("MoveFile", logMsg);

    return TrueMoveFileA(lpExistingFileName, lpNewFileName);
}

// 安装 Hook
void InstallHook()
{
    DetourTransactionBegin();
    DetourUpdateThread(GetCurrentThread());

    DetourAttach(&(PVOID&)TrueCreateFileA, MyCreateFileA);
    DetourAttach(&(PVOID&)TrueCopyFileA, MyCopyFileA);
    DetourAttach(&(PVOID&)TrueDeleteFileA, MyDeleteFileA);
    DetourAttach(&(PVOID&)TrueMoveFileA, MyMoveFileA);

    DetourTransactionCommit();

    WriteLog("SYSTEM", "DLL loaded - file monitoring hooks installed");
}

// 卸载 Hook
void RemoveHook()
{
    DetourTransactionBegin();
    DetourUpdateThread(GetCurrentThread());

    DetourDetach(&(PVOID&)TrueCreateFileA, MyCreateFileA);
    DetourDetach(&(PVOID&)TrueCopyFileA, MyCopyFileA);
    DetourDetach(&(PVOID&)TrueDeleteFileA, MyDeleteFileA);
    DetourDetach(&(PVOID&)TrueMoveFileA, MyMoveFileA);

    DetourTransactionCommit();

    WriteLog("SYSTEM", "DLL unloaded - file monitoring hooks removed");
}

```

```
// DLL 入口点
BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved)
{
    FILE* fp = NULL;  // 将变量声明移到 switch 外面

    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
        // 初始化日志锁
        InitializeCriticalSection(&g_LogLock);
        g_bLogInit = TRUE;

        // 清空旧日志（仅在第一次加载时清空）
        fp = fopen("behavior_log.txt", "w");
        if (fp)
        {
            fprintf(fp, "=== 文件行为监控日志 ===\n");
            fprintf(fp, "监控启动时间: %s\n", __TIME__);
            fclose(fp);
        }

        // 安装钩子
        InstallHook();
        break;

    case DLL_PROCESS_DETACH:
        // 卸载钩子
        RemoveHook();
        // 删除日志锁
        DeleteCriticalSection(&g_LogLock);
        g_bLogInit = FALSE;
        break;
    }
    return TRUE;
}
```

这个代码实现了对 CreateFileA、CopyFileA、DeleteFileA、MoveFileA 的监控，首先编写针对这些 API 函数的 Hook 函数，然后实现了安装和卸载钩子，最后通过 DLLMain 的入口点进行调用。编译命令为

```
cl /LD dllmain.cpp /I D:\detours\include /link /LIBPATH:D:\detours\lib.X64 detours.lib /OUT:hook.dll
```

接下来是 injector 的实现，代码如下所示：

```
// Injector.cpp
```

```c
#include <windows.h>
#include <stdio.h>
#include <string.h>

// 测试函数声明
void RunTests();

int main()
{
    printf("========================================\n");
    printf("  文件行为监控工具 - Injector\n");
    printf("========================================\n\n");

    // 获取当前目录的完整路径
    char dllPath[MAX_PATH];
    GetCurrentDirectoryA(MAX_PATH, dllPath);
    strcat_s(dllPath, "\\hook.dll");

    printf("[1] 检查 DLL 文件...\n");
    if (GetFileAttributesA(dllPath) == INVALID_FILE_ATTRIBUTES)
    {
        printf("[!] 找不到 hook.dll\n");
        printf("[!] 请确保 DLL 文件在当前目录\n");
        printf("按任意键退出...\n");
        getchar();
        return -1;
    }
    printf("[OK] DLL 文件存在: %s\n\n", dllPath);

    // 加载 DLL (这会自动安装钩子)
    printf("[2] 加载 FileMonitor.dll...\n");
    HMODULE hDll = LoadLibraryA(dllPath);
    if (!hDll)
    {
        printf("[!] 加载失败, 错误码: %lu\n", GetLastError());
        printf("[!] 请确保 detours.dll 在当前目录或系统路径中\n");
        getchar();
        return -1;
    }
    printf("[OK] DLL 加载成功, 钩子已安装\n\n");

    printf("[3] 开始监控文件行为...\n");
    printf("    日志文件: behavior_log.txt\n");
    printf("    所有文件操作都会被记录\n\n");
```

```
    // 运行测试
    RunTests();

    // 等待用户输入后卸载
    printf("\n========================================\n");
    printf("按任意键卸载 DLL 并退出...\n");
    getchar();

    printf("\n[4] 卸载 DLL...\n");
    FreeLibrary(hDll);
    printf("[OK] DLL 已卸载，钩子已移除\n");

    printf("\n========================================\n");
    printf("监控完成！请查看 behavior_log.txt\n");
    printf("========================================\n");
    getchar();

    return 0;
}

void RunTests()
{
    printf("\n--- 开始测试 ---\n\n");

    // 测试 1: 创建文件并写入内容
    printf("测试 1: 创建文件 test.txt 并写入内容\n");
    HANDLE hFile = CreateFileA(
        "test.txt",
        GENERIC_WRITE,
        0,
        NULL,
        CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );
    if (hFile != INVALID_HANDLE_VALUE)
    {
        DWORD dwWritten;
        const char* content = "Hello, this is a test file!\n";
        WriteFile(hFile, content, strlen(content), &dwWritten, NULL);
        printf("  已写入内容: %s", content);
        CloseHandle(hFile);
    }
```

```
    else
    {
        printf("  创建文件失败\n");
    }

    Sleep(1000);

    // 测试 2: 复制文件
    printf("\n 测试 2: 复制文件 test.txt -> test_copy.txt\n");
    if (CopyFileA("test.txt", "test_copy.txt", FALSE))
        printf("  复制成功\n");
    else
        printf("  复制失败\n");

    Sleep(1000);

    // 测试 3: 移动/重命名文件
    printf("\n 测试 3: 重命名文件 test_copy.txt -> test_renamed.txt\n");
    if (MoveFileA("test_copy.txt", "test_renamed.txt"))
        printf("  重命名成功\n");
    else
        printf("  重命名失败\n");

    Sleep(1000);

    // 测试 4: 删除移动后的文件
    printf("\n 测试 4: 删除文件 test_renamed.txt\n");
    if (DeleteFileA("test_renamed.txt"))
        printf("  删除成功\n");
    else
        printf("  删除失败\n");

    Sleep(1000);

    // 测试 5: 用记事本打开文件
    printf("\n 测试 5: 用记事本打开 test.txt\n");
    printf("  → 正在启动 notepad.exe 打开文件...\n");
    system("notepad.exe test.txt");

    printf("\n 请关闭记事本窗口继续...\n");
    system("pause");

    printf("\n--- 测试完成 ---\n");
}
```

这部分代码调用 hook.dll 并监控 DLL 中这些函数的调用操作，在控制台输出并将其写入日志。编译命令为 cl Injector.cpp /o Injector.exe。

最终代码运行效果如下所示：

```
========================================
  文件行为监控工具 - Injector
========================================

[1] 检查DLL文件...
[OK] DLL 文件存在: D:\Detours\DetoursHookDemo\hook.dll

[2] 加载 FileMonitor.dll...
[OK] DLL 加载成功, 钩子已安装

[3] 开始监控文件行为...
    日志文件: behavior_log.txt
    所有文件操作都会被记录


--- 开始测试 ---

测试1: 创建文件 test.txt 并写入内容
  已写入内容: Hello, this is a test file!

测试2: 复制文件 test.txt -> test_copy.txt
  复制成功

测试3: 重命名文件 test_copy.txt -> test_renamed.txt
  重命名成功

测试4: 删除文件 test_renamed.txt
  删除成功

测试5: 用记事本打开 test.txt
  → 正在启动 notepad.exe 打开文件...

请关闭记事本窗口继续...
请按任意键继续. . .

--- 测试完成 ---

========================================
按任意键卸载 DLL 并退出...


[4] 卸载 DLL...
[OK] DLL 已卸载, 钩子已移除

========================================
监控完成! 请查看 behavior_log.txt
========================================
```

成功实现了对文件操作行为的监控，最后的日志文件如下：

```
=== 文件行为监控日志 ===
监控启动时间: 10:18:23
[SYSTEM] DLL loaded - file monitoring hooks installed
------------------------------
[FileAccess] 文件: test.txt, 操作: CREATE, 访问类型: WRITE
------------------------------
[CopyFile] 从: test.txt 复制到: test_copy.txt
------------------------------
[MoveFile] 从: test_copy.txt 移动到: test_renamed.txt
------------------------------
[DeleteFile] 删除文件: test_renamed.txt
------------------------------
[SYSTEM] DLL unloaded - file monitoring hooks removed
------------------------------
```

五、实验结论及心得体会

本次实验通过使用 Detours 库完成了对 API 函数的挂钩，并成功基于 Detours 和 R77 完成了 Windows 系统文件操作行为的监控。在实验中我复习了 R77 rootkit 的隐藏行为，并且了解了 Detours 库挂钩 API 的原理，丰富了自身的恶意代码知识。